

NLTK (Natural Language Toolkit) is a popular Python library used for working with human language data (text). It is mainly used for Natural Language Processing (NLP) tasks like text processing, analysis, and building simple language-based applications.

NLTK modules

- tokenize → splits text
- corpus → provides datasets
- stem/lemma → reduces words
- tag → grammar labeling
- parse → sentence structure
- chunk → phrase extraction
- classify → prediction
- metrics → evaluation
- probability → frequency
- semantics → meaning
- util → helper tools

1. nltk.tokenize (Tokenization)

Used to split text into words or sentences.

Common functions:

- word_tokenize
- sent_tokenize
- RegexpTokenizer

2. nltk.corpus (Text datasets)

Provides large text datasets and lexical resources.

Common corpora:

- stopwords
- brown
- movie_reviews
- reuters
- wordnet

3. nltk.stem (Stemming)

Used to reduce words to their root form.

Algorithms:

- PorterStemmer
- LancasterStemmer
- SnowballStemmer

4. nltk.stem.wordnet (Lemmatization)

Used to convert words to their dictionary form.

Example:

- running → run
- better → good

5. nltk.tag (POS Tagging)

Used for Part-of-Speech tagging.

Examples of tags:

- NN (noun)
- VB (verb)
- JJ (adjective)

6. nltk.parse (Parsing)

Used to analyze sentence grammar structure and syntax trees.

7. nltk.chunk (Chunking / NER)

Used for extracting named entities and phrase chunks.

Example:

- “Barack Obama” → PERSON

8. nltk.classify (Classification)

Used for machine learning-based text classification.

Algorithms:

- Naïve Bayes
- Decision Trees

9. nltk.metrics (Evaluation)

Used to evaluate NLP models.

Includes:

- Accuracy
- Precision
- Recall
- F-measure
- Edit distance

10. nltk.probability (Statistical analysis)

Used for frequency and probability analysis.

Includes:

- FreqDist
- ConditionalFreqDist

11. nltk.sem (Semantics)

Used for meaning representation and logical interpretation of text.

12. nltk.util (Utilities)

Provides helper functions for text processing tasks.

Python Features Useful for NLP

1. Strings

Text data in NLP is stored as strings.

```
text = "Python is great for NLP"  
print(text)
```

Output:

Python is great for NLP

2. String Methods

Convert to Lowercase

```
text = "HELLO"  
print(text.lower())
```

Output:

hello

Convert to Uppercase

```
text = "hello"  
print(text.upper())
```

Output:

HELLO

Replace Words

```
text = "I like Java"  
print(text.replace("Java", "Python"))
```

Output:

I like Python

3. Splitting Text

Tokenization begins by splitting text.

```
sentence = "Python for NLP"  
words = sentence.split()
```

```
print(words)
```

Output:

```
['Python', 'for', 'NLP']
```

4. Lists

Lists store multiple tokens.

```
tokens = ["Python", "for", "NLP"]
```

```
print(tokens[0])
```

Output:

Python

5. Loops

Used to process text token by token.

```
words = ["I", "love", "Python"]
```

```
for word in words:  
    print(word)
```

Output:

```
I  
love  
Python
```

6. Dictionaries

Used to store word frequencies.

```
word_count = {  
    "python": 5,  
    "nlp": 3  
}
```

```
print(word_count["python"])
```

Output:

```
5
```

Installing NLP Libraries

Using pip:

```
pip install nltk
```

```
pip install spacy
```

```
pip install transformers
```

First NLP Program in Python

Tokenization using NLTK

```
import nltk  
from nltk.tokenize import word_tokenize
```

```
nltk.download('punkt')
```

```
text = "Python is widely used for NLP."
```

```
tokens = word_tokenize(text)
```

```
print(tokens)
```

Output:

```
['Python', 'is', 'widely', 'used', 'for', 'NLP', '.']
```

Removing Stop Words

Stop words are common words such as:

- is
- am
- the
- a
- an

Example:

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

text = "Python is widely used for NLP"

stop_words = set(stopwords.words('english'))

words = word_tokenize(text)

filtered = []

for word in words:
    if word.lower() not in stop_words:
        filtered.append(word)

print(filtered)
```

Output:

```
['Python', 'widely', 'used', 'NLP']
```

Stemming

Reducing words to their root form.

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

print(stemmer.stem("running"))
print(stemmer.stem("played"))
```

Output:

```
run  
play
```

Sentiment Analysis Example

```
from textblob import TextBlob
```

```
text = TextBlob("I love Python and NLP")
```

```
print(text.sentiment)
```

Output:

```
Sentiment(polarity=0.5, subjectivity=0.6)
```

Polarity = 0.5

- Range: **-1 to +1**
- Indicates **emotion (positive or negative)**

Value	Meaning
+1	Very positive
0	Neutral
-1	Very negative

0.5 means:

Moderately positive sentence

Example:

- "I love Python" → positive polarity

Subjectivity = 0.6

- Range: **0 to 1**
- Indicates **opinion vs fact**

Value	Meaning
0	Objective (fact)
1	Highly subjective (opinion)

0.6 means:

The sentence is more opinion-based than factual

1. nltk.tokenize (Tokenization)

Tokenization is the process of splitting text into words or sentences. It is the first step in NLP.

```
from nltk.tokenize import word_tokenize, sent_tokenize
```

```
text = "Python is great. NLP is powerful."
```

```
print(word_tokenize(text))
```

```
print(sent_tokenize(text))
```

Output:

```
['Python', 'is', 'great', '.', 'NLP', 'is', 'powerful', '.']
```

```
['Python is great.', 'NLP is powerful.']
```

2. nltk.corpus (Stopwords)

Stopwords are common words like *is*, *the*, *am* that are removed during preprocessing.

```
from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words('english'))
```

```
print("is" in stop_words)
```

Output:

```
True
```

3. nltk.stem (Stemming)

Stemming reduces words to their root form by removing suffixes.

```
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
```

```
print(ps.stem("running"))
```

```
print(ps.stem("played"))
```

Output:

```
run
```

```
play
```

4. nltk.stem.wordnet (Lemmatization)

Lemmatization converts words to their dictionary meaning form.

```
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
print(lemmatizer.lemmatize("running", pos="v"))
```

```
print(lemmatizer.lemmatize("better", pos="a"))
```

Output:

```
run
```

```
good
```

5. nltk.tag (POS Tagging)

POS tagging assigns grammatical labels like noun, verb, adjective.

```
from nltk import pos_tag
```

```
from nltk.tokenize import word_tokenize
```

```
text = "Python is a powerful language"
```

```
tokens = word_tokenize(text)
```

```
print(pos_tag(tokens))
```

Output:

```
[('Python', 'NNP'), ('is', 'VBZ'), ('a', 'DT'), ('powerful', 'JJ'), ('language', 'NN')]
```

6. nltk.parse (Grammar Parsing)

Parsing is used to analyze sentence structure (grammar tree).

```
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> "John"
VP -> V NP
V -> "likes"
NP -> "Python"
""")

parser = nltk.ChartParser(grammar)

sentence = ["John", "likes", "Python"]

for tree in parser.parse(sentence):
    print(tree)
```

Output:

```
(S (NP John) (VP (V likes) (NP Python)))
```

7. nltk.chunk (Chunking / NER)

Chunking groups words into meaningful phrases like noun phrases.

```
from nltk import pos_tag, word_tokenize, RegexpParser

text = "Barack Obama was the president"

tokens = word_tokenize(text)
tags = pos_tag(tokens)

grammar = "NP: {<NNP>+}"

cp = RegexpParser(grammar)

result = cp.parse(tags)

print(result)

Output:

(S
  (NP Barack/NNP Obama/NNP)
  was/VBD
```

```
the/DT  
president/NN)
```

8. nltk.classify (Classification)

Used to classify text into categories (positive/negative).

```
from nltk.classify import NaiveBayesClassifier  
  
train_data = [  
    {"love": True, "positive"},  
    {"hate": True, "negative"}  
]  
  
classifier = NaiveBayesClassifier.train(train_data)  
  
print(classifier.classify({"love": True}))
```

Output:

positive

9. nltk.metrics (Accuracy)

Used to measure model performance like accuracy.

```
from nltk.metrics import accuracy  
  
print(accuracy([1,1,0], [1,0,0]))
```

Output:

0.6666666666666666

10. nltk.probability (Frequency Distribution)

Counts how many times words appear.

```
from nltk import FreqDist  
  
text = "python python nlp ai ai ai"  
words = text.split()  
  
fd = FreqDist(words)
```

```
print(fd["ai"])
```

Output:

3

11. nltk.sem (Semantics)

Represents meaning using logical expressions.

```
from nltk.sem import Expression
```

```
read_expr = Expression.fromstring
```

```
expr = read_expr("P & Q")
```

```
print(expr)
```

Output:

(P & Q)

12. nltk.util (N-grams)

Used for helper functions like generating n-grams.

```
from nltk.util import ngrams
```

```
text = "I love NLP"
```

```
tokens = text.split()
```

```
print(list(ngrams(tokens, 2)))
```

Output:

[('I', 'love'), ('love', 'NLP')]